

Compte-rendu 3 : Fonctions

Important :

- Pour limiter le risque d'erreur, pensez à renommer ce fichier sous la forme : CR3-GRP-QUADRINOME suivi de son extension .docx, .odt ou .pdf. *Par exemple, le quadrinome 17 du groupe C qui dépose un fichier PDF nommera son fichier CR3-C-17.pdf*
- Remplissez bien (à nouveau) le nom et numéro de groupe ci-dessous (ex. C-17) et surtout, le prénom et nom des étudiantes et étudiants composant le groupe.
- Seul UN seul des membres du quadrinome doit déposer le compte-rendu. Si plusieurs compte-rendu sont rendus par le même groupe, un seul sera noté (peut-être le plus mauvais...)

Nom ET numéro de groupe : C-08

Prénom	Nom	Adresse e-mail	Pourcentage d'implication dans le travail réalisé*
Sharon	NIXON	sharonnixon835@gmail.com	50%
Abinaya	SASIKARAN	abinayasasikaran@gmail.com	50%
			%
			%

* la somme des pourcentages de cette colonne doit faire 100 % (aux arrondis près)

1 Leçons

1 Capsule-vidéo n°3-a

- Titre de la vidéo : [Déclaration et utilisation des fonctions](#)
- Ce qu'on a appris avec cette vidéo : [Dans cette capsule, j'ai appris à programmer pour calculer l'aire d'un rectangle en utilisant les coordonnées de deux coins opposés, la formule est \$\(x_2 - x_1\) \times \(y_2 - y_1\)\$, et comme le résultat peut être négatif, on vérifie : si l'aire est \$< 0\$, on la rend positive.](#)
- [On a vu comment additionner plusieurs aires pour obtenir une aire totale, en répétant le même calcul pour chaque rectangle](#)

- Pour éviter de recopier le même code plusieurs fois, la vidéo introduit les fonctions. Une fonction sert à regrouper des instructions dans un bloc réutilisable, Par exemple, calculer AireRect (x1, y1, a2) calcule l'aire et renvoie le résultat avec return.
- J'ai appris que pour créer une fonction, il faut:
 - écrire d'abord le nom,
 - ajouter les paramètres,
 - indenter les instructions
 - et terminer par return
- Enfin, on utilise la fonction en lui donnant des arguments dans le bon ordre, ce qui simplifie énormément le programme.
- Exemple de code python illustrant ce qu'on a appris avec cette vidéo :

2 Capsule-vidéo n°3-b

- Titre de la vidéo : Introduction à la programmation Python 3c les procédures
- Ce qu'on a appris avec cette vidéo : Dans cette capsule, j'ai appris ce qu'est une procédure en programmation. Une procédure est similaire à une fonction, mais contrairement à une fonction, elle ne retourne pas de valeur. Elle sert surtout à exécuter des actions, comme afficher un message ou effectuer une opération sans renvoyer de résultat.
- On a vu que les procédures permettent de structurer le programme et éviter de répéter plusieurs fois les mêmes instructions. Elles prennent souvent des paramètres qui permettent de personnaliser leur comportement.
-
- Exemple de code python illustrant ce qu'on a appris avec cette vidéo :

```
def afficheMessage(nom):
    print("Bonjour", nom)

afficheMessage("Sharon")
afficheMessage("Abi")
```

3 Capsule-vidéo n°3-c

- Titre de la vidéo : portée des variables
- Ce qu'on a appris avec cette vidéo : Dans cette capsule, on a appris ce qu'on appelle la portée d'une variable. La portée correspond à la partie du programme dans laquelle une variable peut être utilisée.

Il existe principalement deux types de variables :

- Variables locales : elles sont définies à l'intérieur d'une fonction et ne peuvent être utilisées que dans cette fonction.
- Variables globales : elles sont définies en dehors des fonctions et peuvent être utilisées dans tout le programme.

On a aussi vu que si une variable locale porte le même nom qu'une variable globale, la variable locale sera utilisée à l'intérieur de la fonction.

- Exemple de code python illustrant ce qu'on a appris avec cette vidéo :

```
x = 10 # variable globale
```

```
def test():
```

```
    x = 5 # variable locale
```

```
    print("Dans la fonction :", x)
```

```
test()
```

```
print("Dans le programme principal :", x)
```

On obtient comme résultat :

Dans la fonction : 5

Dans le programme principal : 10

4 Capsule-vidéo n°3-d

- Titre de la vidéo : Bibliothèques

- Ce qu'on a appris avec cette vidéo : Dans cette capsule, j'ai appris ce qu'est une bibliothèque en Python. Une bibliothèque est un ensemble de fonctions déjà programmées que l'on peut utiliser dans son programme pour éviter de tout coder soi-même.
- Pour utiliser une bibliothèque, on doit l'importer avec l'instruction `import`. Cela permet d'accéder à de nombreuses fonctions utiles comme faire des calculs, manipuler des données ou créer des graphiques.
- Par exemple, la bibliothèque `matplotlib` permet de créer des graphiques et des représentations visuelles des données, comme des courbes ou des camemberts.
- Les bibliothèques permettent donc de gagner du temps et d'ajouter facilement des fonctionnalités avancées dans un programme.

- Exemple de code python illustrant ce qu'on a appris avec cette vidéo :

```
import matplotlib.pyplot as plt
```

```
x = [1,2,3,4]
```

```
y = [2,4,6,8]
```

```
plt.plot(x, y)
```

```
plt.title("Exemple de graphique")
```

```
plt.xlabel("x")
```

```
plt.ylabel("y")
```

```
plt.show()
```

5 Capsule-vidéo n°3-e

- Titre de la vidéo :
- Ce qu'on a appris avec cette vidéo :

2 Exercice 2.1 : Visualisation de la composition chimique du corps humain

1 Énoncé

Dans cet exercice, vous allez modifier et compléter le programme de l'exercice précédent.

1. Reprenez le programme que vous avez écrit lors de l'exercice précédent, qui demandait à l'utilisateur la masse totale du corps et le rang de l'élément dont il voulait connaître la masse.
2. Modifiez votre programme pour répartir le code en fonctions et procédures de la manière suivante :
 - une fonction `calculeMasseElement` qui prend en paramètre une masse totale et une proportion d'élément (en pourcentage) et qui retourne (donc avec un `return`, et il ne doit pas y avoir de `print`) la masse calculée.
 - Une procédure `afficheMessageProportion` qui prend en paramètre une proportion d'élément (en pourcentage) et qui ne fait qu'afficher (avec `print`) les messages correspondants (« élément majoritaire », important, faible quantité, minoritaire,...) sans retourner de valeur (donc sans `return`)
 - une procédure `afficheCamembert` qui prend en paramètre un nom d'élément, la proportion de cet élément et qui affiche un camembert avec la proportion de l'élément choisi par rapport à la masse totale du corps. Lors de l'affichage du camembert, seul l'élément choisi par l'utilisateur et le reste des éléments doivent être représentés. L'utilisation de la représentation en camembert (**pie**) de la bibliothèque **matplotlib** est expliqué à la fin de l'exercice. Le camembert doit indiquer
 - Le nom de l'élément sélectionné.
 - La proportion de cet élément par rapport au reste du corps.
3. Dans le bloc principal, la liste proportion sera définie, puis il sera demandé à l'utilisateur la masse totale du corps et le rang de l'élément dont il veut connaître la masse et enfin, la fonction `calculeMasseElement` sera appelée et le résultat récupéré sera affiché, et les procédures `afficheMessageProportion` et `afficheCamembert` seront également appelées pour afficher le message (important, faible, ...) et le camembert.

2 Code

Copier/coller votre code dans le cadre ci-dessous

```
import matplotlib.pyplot as plt
```

```

def calculeMasseElement(masse,prop):
    masseFinal = masse * prop / 100
    return masseFinal

def afficheMessageProportion(prop):
    if prop > 50:
        print("C'est un élément majoritaire dans l'organisme.")
    elif prop > 10:
        print("C'est un élément important dans l'organisme.")
    elif prop > 1:
        print("C'est un élément en faible quantité dans l'organisme.")
    else:
        print("C'est un élément minoritaire dans l'organisme.")

def afficheCamembert(nomElement, prop):
    plt.figure(figsize=(6,6))
    plt.pie(
        [prop, 100 - prop],
        labels=[nomElement, "Autres éléments"],
        autopct='%1.1f%%',
        startangle=90,
        colors=['#ffc7f4', '#a7a8eb'],
        explode=(0.1, 0)
    )
    plt.title(f"Proportion de {nomElement} dans le corps")
    plt.axis('equal')
    plt.show()

proportions = [['Oxygène', 'O', 65], ['Carbone', 'C', 18.5], ['Hydrogène', 'H', 9.5],
['Azote', 'N', 3.3], ['Calcium', 'Ca', 1.5], ['Phosphore', 'P', 1], ['Potassium', 'K',
0.4], ['Soufre', 'S', 0.3], ['Sodium', 'Na', 0.2], ['Chlore', 'Cl', 0.2], ['Magnésium',
'Mg', 0.1]]

masse = float(input("Entrez la masse totale du corps de la personne (en kg) : "))
rang = int(input("Entrez le rang de l'élément chimique : "))

if rang < 1 or rang > len(proportions):
    print("Le rang saisi dépasse le nombre d'éléments disponibles.")
else :
    nomElement = proportions[rang - 1][0]
    prop = proportions[rang - 1][2]
    masseElementF = calculeMasseElement(masse, prop)
    afficheMessageProportion(prop)
    print("La masse de", nomElement, "dans le corps est de", masseElementF , "kg.")

```

```
afficheCamembert(nomElement, prop)
```

3 Résultat

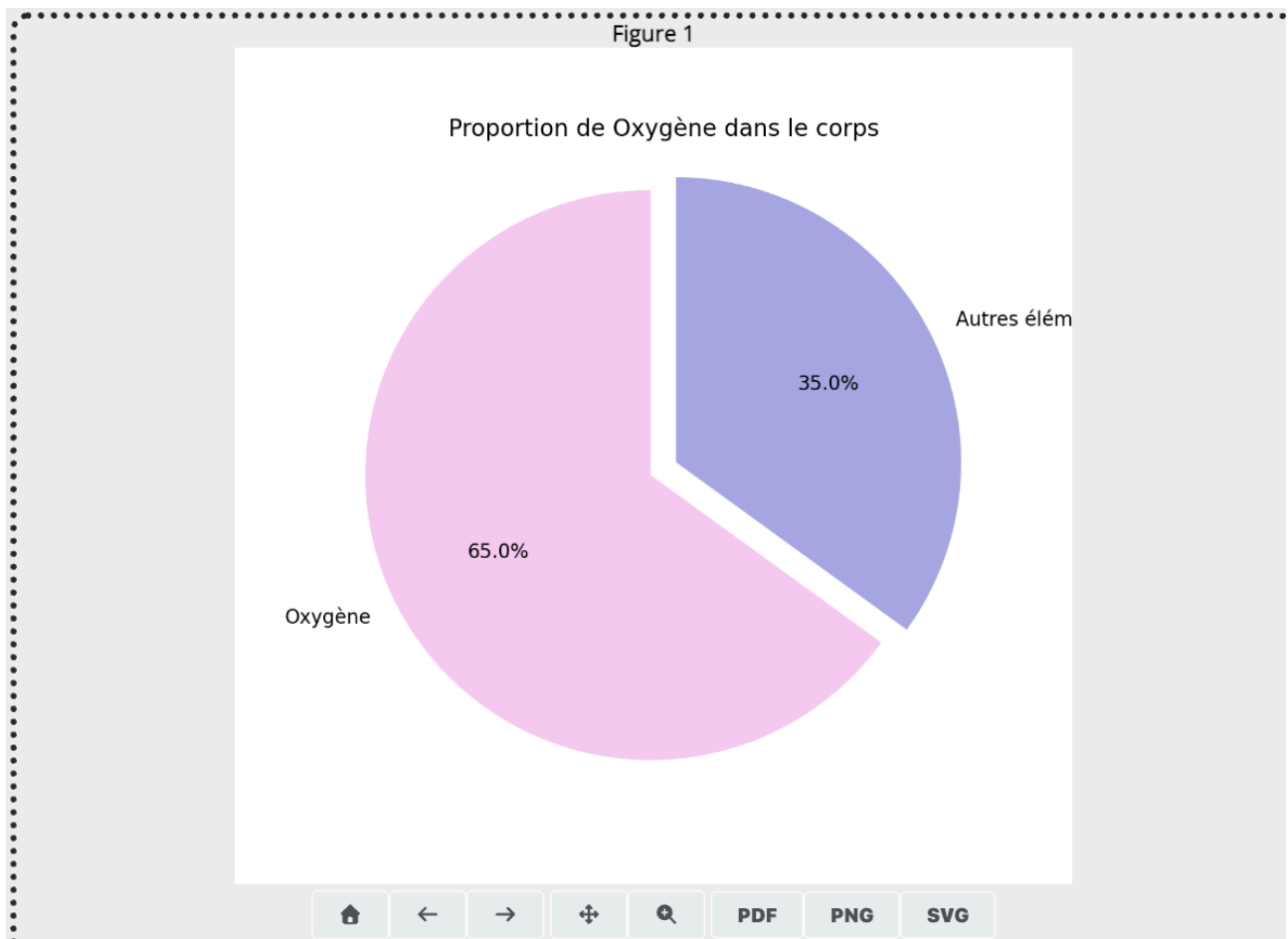
Test 1

Description du test (quels paramètres, que cherche-t-on à tester) :

Pour ce test, on entre une masse totale du corps de 57 kg et on choisit le rang 1 correspondant à l'oxygène (65 %).

Ce test permet de vérifier si la fonction `calculeMasseElement` calcule correctement la masse d'un élément majoritaire dans le corps humain et si le programme affiche correctement le message correspondant ainsi que le camembert.

```
Python 3.12.1 (main, Nov 15 2024 14:17:00)
Type "help", "copyright", "credits" or "license" for more i
nformation.
>>> # script executed
Entrez la masse totale du corps de la personne (en kg) : 57
Entrez le rang de l'élément chimique : 1
C'est un élément majoritaire dans l'organisme.
La masse de Oxygène dans le corps est de 37.05 kg.
>>>
```



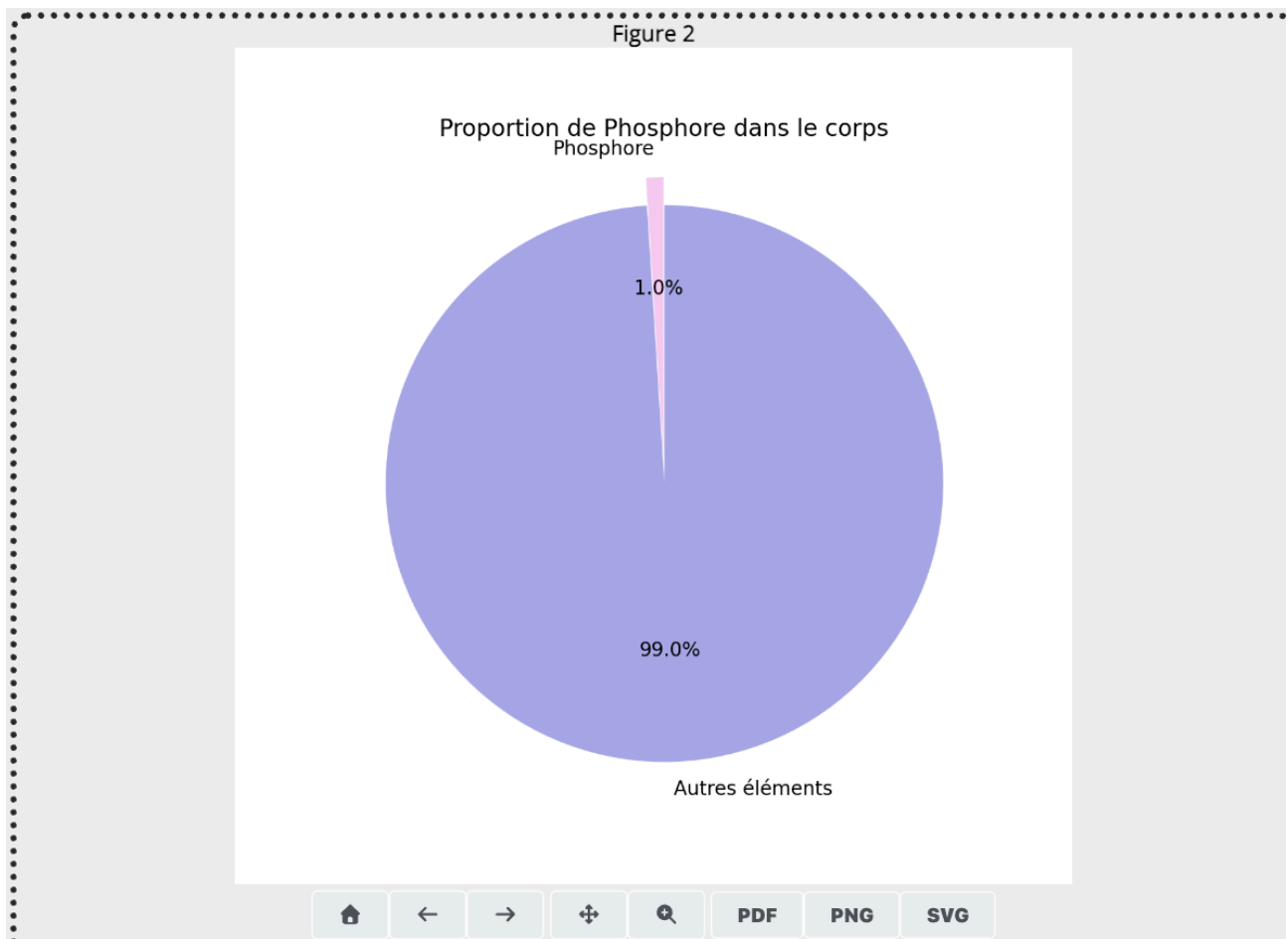
Test 2

Description du test (quels paramètres, que cherche-t-on à tester) :

Pour ce test, on entre une masse totale du corps de 80 kg et on choisit le rang 6 correspondant au phosphore (1 %).

Ce test permet de vérifier si le programme gère correctement un élément présent en très faible quantité dans le corps et si le message « élément minoritaire » s'affiche correctement. Il permet aussi de vérifier que le camembert représente correctement la proportion de cet élément par rapport au reste du corps.

```
...
Entrez la masse totale du corps de la personne (en kg) : 80
Entrez le rang de l'élément chimique : 6
C'est un élément minoritaire dans l'organisme.
La masse de Phosphore dans le corps est de 0.8 kg.
>>>
```



4 Réflexion

Quels éléments du cours actuel a fait intervenir cet exercice ?

- Cet exercice a permis d'utiliser les fonctions et les procédures en Python.

Nous avons appris à séparer un programme en plusieurs parties pour le rendre plus clair et plus réutilisable. Une fonction a été utilisée pour calculer la masse d'un élément chimique, tandis que des procédures ont été utilisées pour afficher des messages et représenter les résultats sous forme de graphique. Nous avons également utilisé la bibliothèque matplotlib pour créer un camembert représentant la proportion d'un élément dans le corps humain.

Quels éléments des cours précédents a fait intervenir cet exercice ?

- Cet exercice a aussi utilisé des notions vues dans les chapitres précédents comme les variables, les listes, les conditions (if / elif / else) et les entrées utilisateur avec input. Nous avons également utilisé des opérations mathématiques simples pour calculer la masse d'un élément à partir d'une proportion.

3 Exercice 2.2 : Calcul de la croissance d'une population bactérienne avec une limite environnementale

1 Énoncé

Dans cet exercice, vous allez transformer le programme que vous avez écrit précédemment en une fonction générique. Vous aurez également à afficher la courbe représentant les résultats obtenus en fonction du temps d'observation.

1. **Reprenez votre code** de l'exercice précédent où vous avez modélisé la croissance bactérienne qui double à chaque cycle jusqu'à atteindre la capacité maximale `N_max` puis meurt suivant un taux de mortalité de `M` % à chaque cycle, **et transformez votre programme en une fonction que vous appellerez**

`calculeNbBacteries`. Votre fonction doit prendre en paramètre :

- `N0` : le nombre initial de bactéries,
- `t_cycle` : le temps nécessaire pour un cycle en minutes,
- `N_max` : la capacité maximale de l'environnement (la population maximale possible),
- `M` : le taux de mortalité des bactéries en pourcentage une fois la capacité maximale atteinte,
- `temps_total` : le temps total d'observation en minutes.

La fonction devra calculer et retourner le nombre de bactéries lorsque le `temps_total` d'observation est atteint.

- Demandez à l'utilisateur de rentrer un nombre initial de bactéries, le temps d'un cycle, la capacité maximale de l'environnement et le taux de mortalité.
- Vous devrez ensuite faire varier `temps_total` en utilisant les valeurs suivantes : 0, 250, 500, 750, 1000, 1250 et 1500 minutes, et observer l'évolution de la population à chaque instant. Pour cela, vous appellerez votre fonction `calculeNbBacteries` depuis votre bloc d'instruction principal avec les 6 valeurs indiquées pour le `temps_total`.
- **Utilisation de `matplotlib.pyplot`** pour tracer la courbe représentant l'évolution de la population au cours du temps. L'axe des x sera le temps (`temps_total`) et l'axe des y représentera la taille de la population bactérienne.

2 Code

Copier/coller votre code dans le cadre ci-dessous

```
import matplotlib.pyplot as plt

def calculeNbBacteries(N0, tempsCycle, N_max, M, tempsTotal):
    temps = 0
    capacite_atteinte = False
```

```

while temps < tempsTotal:
    temps += tempsCycle

    if not capacite_atteinte:
        N0 *= 2

        if N0 >= N_max:
            N0 = N_max
            capacite_atteinte = True
        else:
            N0 *= (1 - M / 100)

    return N0

def afficheGraphe(valeursTemps, population):
    plt.figure(figsize=(8,6))
    plt.plot(valeursTemps, population, marker='o', color='purple', linestyle='-')
    plt.title("Nombre de bactéries en fonction du temps")
    plt.xlabel("Temps (minutes)")
    plt.ylabel("Nombre de bactéries")
    plt.grid(True)
    plt.show()

# Input
N0 = int(input("Entrez le nombre initial de bactéries : "))
tempsCycle = int(input("Entrez le temps nécessaire pour un cycle (en minutes) : "))
N_max = int(input("Entrez la capacité maximale de l'environnement : "))
M = float(input("Entrez le taux de mortalité (en pourcentage) : "))

valeursTemps = [0, 250, 500, 750, 1000, 1250, 1500]
population = []

for tempsTotal in valeursTemps:
    nombreB = calculeNbBacteries(N0, tempsCycle, N_max, M, tempsTotal)
    population.append(nombreB)
    print(f"Après {tempsTotal} minutes, la population bactérienne est d'environ {nombreB:.0f} bactéries. ")

afficheGraphe(valeursTemps, population)

```

3 Résultat

Test 1

Description du test (quels paramètres, que cherche-t-on à tester) :

Pour ce test, on utilise les paramètres suivants :

- population initiale : 500 bactéries
- temps d'un cycle : 30 minutes
- capacité maximale : 1000 bactéries
- taux de mortalité : 10 %

Ce test permet de vérifier si la fonction `calculeNbBacteries` calcule correctement l'évolution de la population bactérienne au cours du temps. Il permet aussi de vérifier que la population double jusqu'à atteindre la capacité maximale, puis diminue selon le taux de mortalité. Le graphique permet de visualiser cette évolution.

information.

```
>>> # script executed
```

```
Entrez le nombre initial de bactéries : 500
```

```
Entrez le temps nécessaire pour un cycle (en minutes) : 30
```

```
Entrez la capacité maximale de l'environnement : 1000
```

```
Entrez le taux de mortalité (en pourcentage) : 10
```

```
Après 0 minutes, la population bactérienne est d'environ 500 bactéries.
```

```
Après 250 minutes, la population bactérienne est d'environ 430 bactéries.
```

```
Après 500 minutes, la population bactérienne est d'environ 185 bactéries.
```

```
Après 750 minutes, la population bactérienne est d'environ 80 bactéries.
```

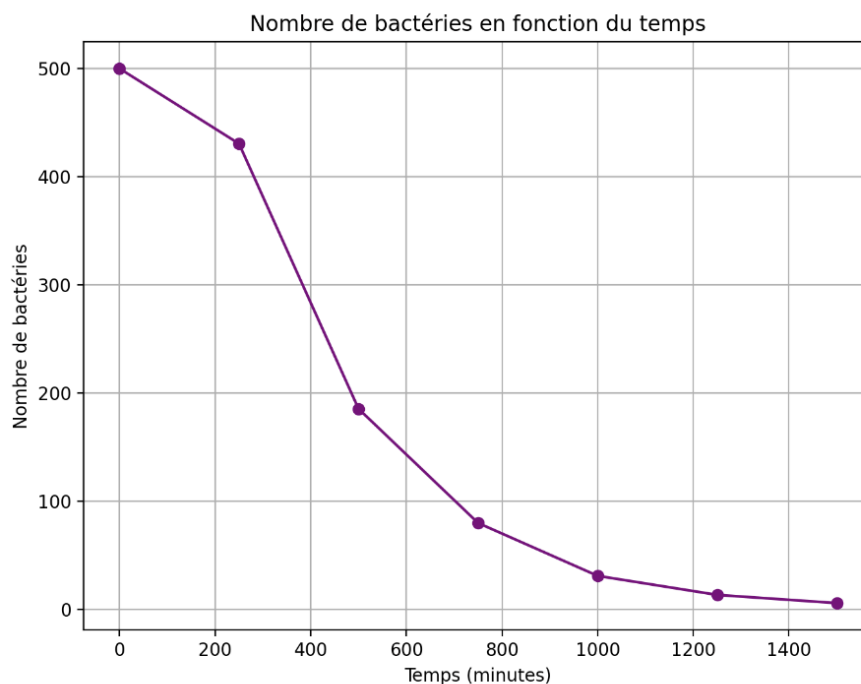
```
Après 1000 minutes, la population bactérienne est d'environ 31 bactéries.
```

```
Après 1250 minutes, la population bactérienne est d'environ 13 bactéries.
```

```
Après 1500 minutes, la population bactérienne est d'environ 6 bactéries.
```

```
>>>
```

Figure 1



ateur doivent être visibles. **Pensez à bien mettre également le graphique généré.**

4 Réflexion

Quels éléments du cours actuel a fait intervenir cet exercice ?

- Cet exercice utilise les fonctions en Python pour rendre le programme plus organisé et réutilisable. La fonction `calculeNbBacteries` permet de calculer la population bactérienne en fonction de plusieurs paramètres. Nous avons aussi utilisé une fonction pour tracer un graphique afin de représenter l'évolution de la population au cours du temps avec la bibliothèque `matplotlib`.

Quels éléments des cours précédents a fait intervenir cet exercice ?

- Cet exercice utilise également des notions vues auparavant comme les boucles `while`, les conditions `if`, les variables, les listes, et les entrées utilisateur avec `input`. Ces éléments permettent de simuler l'évolution d'une population bactérienne au fil du temps et de stocker les résultats afin de les afficher dans un graphique.

4 Conclusion

Que devons-nous retenir d'important sur ce chapitre de cours ? Qu'y avons nous appris ?

- Dans ce chapitre, nous avons appris à structurer un programme en utilisant des fonctions et des procédures. Cela permet de rendre le code plus clair, plus organisé et plus facile à réutiliser. Nous avons également appris à utiliser des paramètres et des valeurs de retour pour transmettre des informations entre les différentes parties du programme.

- Nous avons aussi découvert comment utiliser des bibliothèques Python, notamment matplotlib, pour représenter des données sous forme de graphiques. Cela permet de mieux comprendre les résultats d'un programme, par exemple en visualisant la composition chimique du corps humain ou l'évolution d'une population bactérienne dans le temps.